



عملي

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 09/09/2020

كلية الهندسة المعلوماتية

السنة الثالثة

ملحق قواعد المعطيات 1

عملي مشترك



قواعد معطيات

السلام عليكم ورحمة الله وبركاته

File System

File System: أسلوب تخزين البيانات على شكل ملفات منفصلة ومجلدات، وبرامج خاصة تقرأ وتكتب داخل الملفات مباشرة، كل برنامج يتعامل مع ملفاته.

هذا الأسلوب كان شائعاً قبل ظهور DBMS الحديثة، وما يزال مستخدماً في بعض الأنظمة الصغيرة، لكنه يصبح مشكلة عندما تكبر البيانات.

مشاكل File System:

1. Data Redundancy

- من أشهر مشاكل File System ، وتحدث عندما تكون البيانات مخزنة في أكثر من ملف أو أكثر من برنامج.
- تؤدي هذه المشكلة لهدر مساحة التخزين، زيادة تكلفة التخزين، وصعوبة الصيانة.

2. Data Inconsistency

- نتيجة مباشرة لتكرار البيانات، إذا تغيرت بيانات عنصر في ملف، يجب أن تتغير في جميع الملفات الأخرى المرتبطة بهذا العنصر.

- الحل المنطقي: تحتاج أن تكون البيانات مخزنة بطريقة تضمن وجود مصدر واحد للمعلومة، بحيث يتم التحديث أو الحذف على مصدر واحد تلقائياً، مع وجود قيود على التكامل وعلاقات واضحة بين الملفات.

3. Difficult Data Access

- في نظام الملفات التقليدي، الوصول للبيانات يكون مرتبطاً ببرنامج محدد.

مثال :

أعطني أسماء الروايات التي اشتريتها في آخر 5 أشهر، وكتبها نجيب محفوظ ورتبها أبجدياً .
 في File System قد تحتاج إلى :
 • فتح عدة ملفات
 • كتابة منطق دمج بينها
 • فلترة وفرز يدوي أو برمجى معقد

4. Difficult Data Process

- المعالجات التقليدية _ كالإضافة والحذف والتعديل _ صعبة جداً وتستغرق وقتاً طويلاً للتنفيذ في حال ازدياد حجم النظام.

الحل: توفر قواعد البيانات لغات استعلام مثل SQL، وتكون العمليات الأساسية معرفة وجاهزة للتنفيذ بسهولة.

5. Program-Data Dependence

- يكون البرنامج مكتوباً على أساس شكل الملف مباشرة. إذا تغير شكل الملف، تضطر لتعديل كل البرامج المرتبطة به.

6. Integrity Problems

- توجد قواعد يجب احترامها. في الـ File System تكون هذه القواعد داخل البرامج نفسها مما يسهل نسيانها أو تطبيقها بشكل متضارب.

الحل :تتيح قواعد البيانات فرض القيود مركزياً

7. Security Problems

- الصلاحيات في الـ File System تكون على مستوى الملف أو المجلد بالكامل

الحل :توفر أنظمة قواعد البيانات إدارة دقيقة للصلاحيات والأدوار مع ميزة المراقبة والتدقيق.

8. Concurrent Access Problems

- إذا حاول أكثر من مستخدم تعديل نفس البيانات في نفس الوقت تحدث مشاكل.

الحل :توفر قواعد البيانات آليات احتراافية لإدارة التزامن

9. Backup and Recovery Problems

- التعافي من الأعطال صعب جداً في الـ File System لعدم وجود آليات جاهزة لحفظ النقاط الزمنية.
- الحل: توفر قواعد البيانات سجلات التعديلات والنسخ الاحتياطي للتعافي التلقائي والسهل

10. صعوبة بناء علاقات بين البيانات Relations Management Problems

- البيانات في الواقع مترابطة. في الـ File System ثدار العلاقات يدوياً عبر مطابقة الـ IDs برمجياً مما يزيد الأخطاء.
- الحل: قواعد البيانات (خاصة العلائقية) مصممة أساساً لبناء ودعم العلاقات.

نظام الملفات (File System)	نظام إدارة قواعد البيانات (DBMS)
تكرار البيانات	تقليل التكرار
عدم الاتساق	تحسين الاتساق
صعوبة الوصول والاستعلام	لغة استعلام قوية (SQL)
اعتماد البرامج على شكل الملفات	استقلالية البيانات (Data Independence)
صعوبة فرض القيود	فرض القيود على مستوى البيانات مركزياً
ضعف الأمان	أمان أعلى وصلاحيات دقيقة (Roles)
ضعف التزامن	إدارة التزامن (Locks, Transactions)
صعوبة الاسترجاع	أنظمة نسخ احتياطي واسترجاع متطورة (Backup/Recovery)
صعوبة بناء علاقات بين البيانات	تصميم مناسب للاستيعاب كافة العلاقات
صعوبة التوسع والصيانة	مركزية وتنظيم أفضل

مفاهيم أساسية في قواعد البيانات

- قاعدة البيانات (Database): مجموعة منظمة من البيانات المترابطة، تخزن في الأنظمة الحاسوبية بطريقة تسمح بالوصول إليها وإدارتها وتحديثها بسهولة وكفاءة.
- نظام إدارة قواعد البيانات (DBMS): البرنامج أو النظام المستخدم لإنشاء وإدارة والتحكم في قواعد البيانات (إدخال، تحديث، حذف، حماية، صلاحيات، وتنفيذ الاستعلامات Queries).

أقسام أنظمة قواعد البيانات (DBMS Types)

قواعد البيانات العلائقية (RDBMS): يتم التعامل معها بلغة الاستعلام الهيكلية SQL، ومن أمثلتها:

1. MySQL
2. Microsoft SQL Server
3. Oracle Database
4. PostgreSQL

قواعد البيانات غير العلائقية (NoSQL): وتشمل أنواعاً مثل:

1. Key-Value Stores
 2. Document Databases
 3. Column-Family Databases
 4. Graph Databases
- أمثلة عليها: MongoDB, Cassandra, Redis

مخطط العلاقات الكيانية (ERD: Entities Relationship Diagrams)

يستخدم لتمثيل وتصميم قواعد البيانات بصرياً، يساعد في تصور تنظيم البيانات داخل النظام وكيفية ارتباطها ببعضها قبل التنفيذ الفعلي، وذلك لتجنب الأخطاء وتسهيل التواصل بين أعضاء فريق التطوير.

■ مكونات الـ ERD الأساسية:

1. **الكيانات (Entities):** هي المفاهيم أو العناصر الرئيسية التي يدور حولها النظام ونحتاج لحفظ بياناتها (تمثل فئة عامة وليس عنصراً واحداً) (مثال: مستشفى: المريض، الطبيب، الغرفة).
2. **الخصائص (Attributes):** هي المعلومات أو البيانات التي تصف الكيان وتوضح تفاصيله وتعطيه قيمة (مثل: الاسم، الرقم الجامعي، التخصص لـ كيان "الطالب").

3. **العلاقات (Relationships)** : هي الروابط والاتصالات التي تجمع بين كيانات أو أكثر في النظام، وتوضح كيفية تفاعل هذه الكيان مع بعضها البعض (مثال: الطبيب يعالج المريض، الطالب يسجل في المادة، الموظف يعمل في القسم).

أنواع الAttribute

1. Simple Attribute (الخاصية البسيطة)

هي خاصية لا يمكن تقسيمها إلى أجزاء أصغر.

مثال:

الجنس - راتب الموظف..

يعني قيمة واحدة فقط وواضحة وغير قابلة للتجزئة.

2. Attribute Key

هو خاصية تستخدم للتعريف أو التمييز داخل الكيان.

الفرق بين PK و FK

Primary Key (PK) المفتاح الأساسي

هو خاصية تميز كل سجل بشكل فريد.

لا يتكرر ولا يكون فارغ (NULL).

مثال: رقم الطالب في جدول الطلاب.

Foreign Key (FK) المفتاح الخارجي

هو خاصية تربط بين جدولين.

يشير إلى ال Primary Key في جدول آخر.

يستخدم لعمل العلاقات بين الكيانات.

مثال:

جدول الطلاب (Student)

جدول التسجيل (Enrollment)

رقم الطالب في Enrollment يكون FK لأنه يشير إلى PK في Student

3. Derived Attribute (خاصية مشتقة)

هي خاصية لا يتم تخزينها في قاعدة البيانات، بل تحسب من خصائص أخرى.

مثال:

العمر ← يُحسب من تاريخ الميلاد
يعني نحسبها من بيانات موجودة.

4. Composite Attribute (خاصية مركبة)

هي خاصية يمكن تقسيمها إلى أجزاء أصغر.

مثال:

الاسم الكامل → (اسم أول + اسم أب + اسم عائلة)
العنوان → (مدينة + شارع + رقم منزل)

تكوّن من أكثر من جزء.

5. Multivalued Attribute (خاصية متعددة القيم)

هي خاصية يمكن أن تأخذ أكثر من قيمة لنفس الكيان.

مثال:

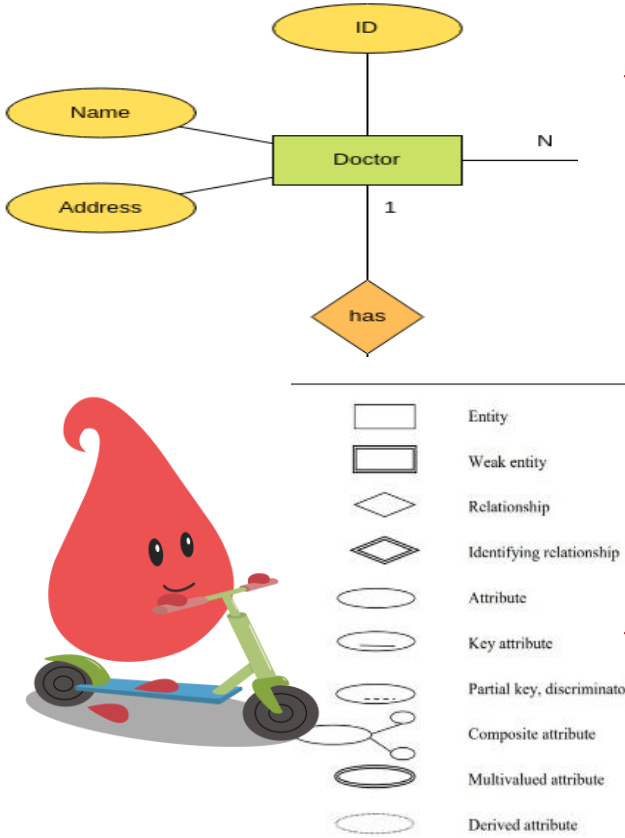
أرقام الهاتف (أكثر من رقم لشخص واحد)

يعني ليس لها قيمة واحدة فقط.

■ أنواع تحديد التعددية (Cardinality Types):

1. **واحد لواحد (1:1) One-to-One**: كل عنصر يرتبط بعنصر واحد فقط من الطرف الآخر (مثال: شخص ↔ هوية).
2. **واحد لمتعدد (1:M) One-to-Many**: عنصر واحد من الكيان الأول يرتبط بعدة عناصر من الثاني، وعنصر الثاني يرتبط بعنصر واحد فقط من الأول (مثال: قسم ↔ موظفين، وهو النوع الأكثر شيوعاً).
3. **متعدد لمتعدد (M:N) Many-to-Many**: عدة عناصر من الطرف الأول ترتبط بعدة عناصر من الطرف الثاني (مثال: طالب ↔ مادة، أو منتج ↔ طلب).

طرق رسم وتوثيق مخططات العلاقات الكيانية (ERD)

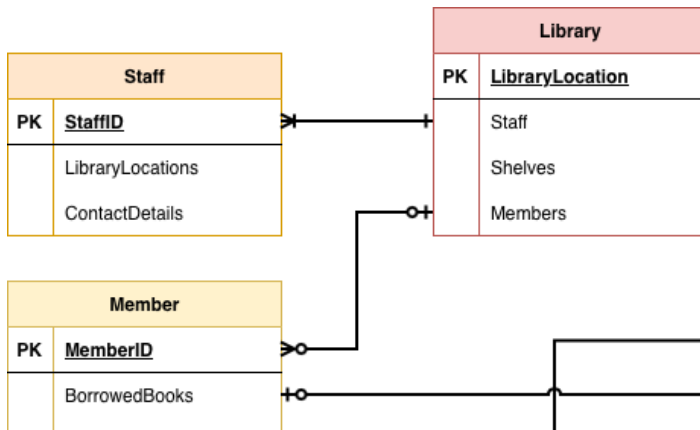


Chain Notation

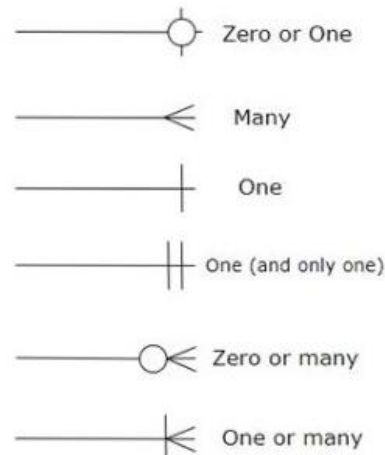
هو تدوين مفاهيمي (منطقي) يستخدم في المراحل الأولى للتصميم أو للأغراض الأكاديمية. يركز هذا التدوين على القواعد المنطقية للعلاقات بين الكيانات باستخدام نصوص صريحة مثل الرموز 1 و N التي تكتب وتوضع فوق خطوط العلاقات.

Crow's Foot

هو تدوين فيزيائي (تنفيذي) يركز بشكل أكبر على تفاصيل التنفيذ البرمجي وقيم الحد الأدنى والحد الأقصى للمشاركة. يُعد التدوين القياسي المستخدم في معظم أدوات ومصمّمات قواعد البيانات الحديثة؛ نظراً لأنه يسهل فهم الآلية الفيزيائية لربط المفاتيح الخارجية وكيفية بناء وتنفيذ الجداول فعلياً في النظام.



شد حيلك يا كتكوت:



الجبر العلاقائي (Relational Algebra)

لغة استعلام إجرائية، نقوم بتطبيقها على الجداول بعلاقاتها، لنحصل على نتائج (سجلات) محددة نريدها. ويمثل أساساً نظرياً مجرداً لعلاقات قواعد البيانات ولغة SQL، وهو ليس تعليمات SQL. بل هو المنطق الرياضي الذي يعمل خلف الكواليس لتحديد كيفية الوصول للبيانات.

لدينا نوعان من العمليات:

1. عمليات أحادية Unary: تعمل على جدول واحد فقط

1. عملية الاختيار (σ Select)

الوظيفة: فلتر صفوف الجدول بناءً على شرط منطقي محدد

مثال: جلب موظفي قسم ال IT عبر التعبير: $\sigma_{dept='IT'}(Employee)$

Salary	Dept	Name	EID
5000	IT	Ahmed	1
4500	HR	Sara	2
6000	IT	Ali	3

$\sigma_{dept='IT'}(Employee)$

Salary	Dept	Name	EID
5000	IT	Ahmed	1
6000	IT	Ali	3

2. عملية الإسقاط (Π Project)

الوظيفة: اختيار واختزال أعمدة معينة فقط من الجدول وإهمال باقي الحقول.

مثال: استخراج أسماء الموظفين ورواتبهم فقط: $\Pi_{Name,Salary}(Employee)$

Salary	Dept	Name	EID
5000	IT	Ahmed	1
4500	HR	Sara	2
6000	IT	Ali	3

$\Pi_{Name,Salary}(Employee)$

Salary	Name
5000	Ahmed
4500	Sara
6000	Ali

3. عملية إعادة التسمية (ρ Rename)

الوظيفة: تغيير اسم الجدول أو أسماء الأعمدة لتجنب اللبس والغموض، خاصة عند دمج الجدول مع نفسه.

(مثال: $\rho_{Staff}(Employee)$)

2. عمليات المجموعات (Set Operations): تُستخدم هذه العمليات لدمج أو مقارنة السجلات بين جدولين، وتعتمد بشكل أساسي على قواعد المجموعات الرياضية.

■ قاعدة "توافق الاتحاد" (Union Compatibility / Type Compatibility): هي الشرط الأساسي الذي لا يمكن دونه إجراء أي عملية من عمليات المجموعات (الاتحاد، التقاطع، الفرق) بين جدولين. لكي يكون الجدولان متوافقين، يجب أن يتحقق شرطان حتماً:

1. اتساوي عدد الأعمدة بين الجدولين.
2. توافق النطاقات (Domains): أي أن أنواع البيانات (Data Types) في الأعمدة المتقابلة يجب أن تكون متطابقة.

سنعتمد على الجدولين لتوضيح العمليات:

Science Top		Math Top	
Name	Student_ID	Name	Student_ID
Ahmed	101	Sara	102
Sara	102	Mona	104
Ali	103	Omar	105

أولاً: عملية الاتحاد (Union U): دمج جميع السجلات المتواجدة في الجدول الأول مع السجلات المتواجدة في الجدول الثاني في جدول خرج Science Top U Math Top:

Name	Student_ID
Ahmed	101

Name	Student_ID
Sara	102
Ali	103
Mona	104
Omar	105

نلاحظ أن الطالبة (Sara) التي تحمل الرقم (102) كانت متواجدة في كلا الجدولين الأساسيين، وعند تطبيق عملية الاتحاد ظهرت في جدول الخرج مرة واحدة فقط، حيث يقوم الجبر العلاقتي بحذف التكرار تلقائياً.

ثانياً: عملية التقاطع ($\cap$): استخراج السجلات المشتركة فقط، أي الصفوف التي تتطابق قيمها بالكامل وتواجد في الجدول الأول والجدول الثاني في آن واحد $\text{Science Top} \cap \text{Math Top}$

Name	Student_ID
Sara	102

ثالثاً: عملية الفرق ($-$): طرح سجلات جدول معين من جدول آخر؛ حيث تستخرج السجلات والعناصر التي تنتمي وتتواجد في الجدول الأول (الأيسر)، بشرط ألا يكون لها أي وجود أو مثيل في الجدول الثاني (الأيمن).

ثانياً: $\text{Math Top} - \text{Science Top}$

Name	Student_ID
Mona	104
Omar	105

أولاً: $\text{Science Top} - \text{Math Top}$

Name	Student_ID
Ahmed	101
Ali	103

الجداء الديكارتي والدمج (Cartesian Product & Join)

سنستخدم الجدولين للشرح:

2. جدول المشاريع (Project table)

EID (Manager)	PName	PID
1	System X	101
3	Mobile App	102

1. جدول الموظفين (Employee table)

Salary	Dept	Name	EID
5000	IT	Ahmed	1
4500	HR	Sara	2
6000	IT	Ali	3

أولاً: الجداء الديكارتي ($\times$ Cartesian Product): قوم بربط كل صف في الجدول الأول بكل صف في الجدول الثاني بشكل كامل وبدون أي شروط

مثال: $\text{Employee} \times \text{Project}$

EID (Manager)	PName	PID	Salary	Dept	Name	EID
1	System X	101	5000	IT	Ahmed	1
3	Mobile App	102	5000	IT	Ahmed	1
1	System X	101	4500	HR	Sara	2
3	Mobile App	102	4500	HR	Sara	2
1	System X	101	6000	IT	Ali	3
3	Mobile App	102	6000	IT	Ali	3

ثانياً: عملية الدمج / الربط ($\bowtie$ Join): العملية الأكثر استخداماً في قواعد البيانات، وتقوم بدمج البيانات من جدولين بناءً على حقل مشترك بينهما فقط (مثل ربط الموظف بمشروعه).

مثال $\text{Employee} \bowtie_{\text{EID=Manager}} \text{Project}$

PName	PID	Salary	Dept	Name	EID
System X	101	5000	IT	Ahmed	1
Mobile App	102	6000	IT	Ali	3

العمليات المركبة (Compound Operations):

تستخدم عندما يتطلب الاستعلام تطبيق أكثر من عملية جبرية (دمج، فلترة، إسقاط) للوصول إلى النتيجة النهائية.

مثال: اكتب التعبير الذي يستخرج أسماء الموظفين الذين يعملون في قسم الـ IT ويديرون مشروعاً.

الحل:

1. ربط الجداول: نقوم أولاً بدمج جدول الموظفين مع المشاريع بناءً على الحقل المشترك ونعطي اسماً مؤقتاً للعلاقة

$$R_1 = \text{Employee} \bowtie_{(\text{EID}=\text{Manager})} \text{Project}$$

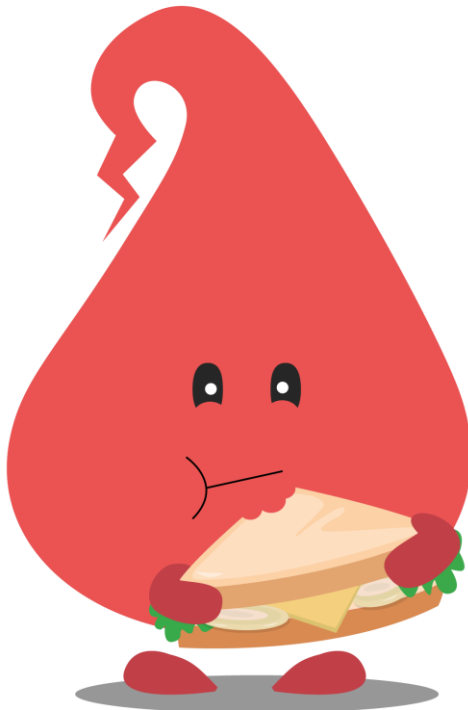
2. فلترة الصفوف (الشرط): نختار من العلاقة السابقة الموظفين الذين ينتمون لقسم الـ IT فقط، ونعطي اسماً للعلاقة

$$R_2 = \sigma_{(\text{dept}='IT')}(R_1)$$

3. إسقاط الأعمدة: نختار حقل الأسماء المطلوب فقط من العلاقة الأخيرة:

$$\Pi_{\text{Name}}(R_2)$$

التعبير النهائي المدمج بالكامل: $\Pi_{\text{Name}}(\sigma_{(\text{dept}='IT')}(\text{Employee} \bowtie_{(\text{EID}=\text{Manager})} \text{Project}))$



وكم من مشكلة حلها

سندويشة مكدوس

وكاسة شاي

Relational Database - RDBI قواعد البيانات العلائقية

Rows and Columns

هي نوع من قواعد البيانات يقوم بتخزين البيانات بطريقة منظمة داخل جداول Tables.

الجدول هو بنية تستخدم لتنظيم البيانات، ويتكون من:

- **أعمدة (Columns):** تمثل نوع البيانات مثل الاسم، العمر، البريد الإلكتروني
- **صفوف (Rows):** تمثل السجلات أو البيانات الفعلية مثل بيانات شخص ما

كل جدول يحتوي على بيانات مرتبطة بموضوع محدد ويمكن ربط الجداول ببعضها باستخدام علاقات Relationships وهذا ما يجعلها "علائقية".

ملاحظة: تسمى الصفوف (Records) أو (Tuples)، وتمثل سجلات كاملة لبيانات معينة.

كل صف يمثل سجل كامل المعلومة معينة، مثل بيانات شخص واحد.



جدول Persons:

id	Name	SurName	Age
1	Jodie	Tucker	34
2	Joyden	Archer	56
3	Groce	Wheeler	18
4	Freddie	Humphries	56

ملاحظة: تسمى الأعمدة (Fields) أو (Attributes)، وتمثل خصائص البيانات في الجدول.

كل عمود يمثل خاصية للبيانات، مثل معلومة معينة للسجلات المخزنة.

Cell: هي تقاطع صف Row مع عمود Column ، وتسمى أيضا Value أو Data Value.

SQL (Structured Query Language)

هي لغة تستخدم لإدارة ومعالجة البيانات داخل أنظمة إدارة قواعد البيانات العلائقية RDBMS

تتيح SQL للمستخدمين تعريف هيكل قاعدة البيانات (الجدول والعلاقات)، وإدارة البيانات داخلها (استرجاع إضافة تحديث حذف)

تعتمد قواعد البيانات العلائقية على جداول تتكون من صفوف Rows وأعمدة Columns
تعمل SQL مع أنظمة متعددة مثل MySQL، PostgreSQL، SQL Server، Oracle

كتابة استعلامات - SQL (Sql Query)

أي استعلام SQL يتكون من نوعين من الكلمات:

- كلمات محجوزة (Keywords) وهي أوامر SQL الأساسية.
- أسماء كائنات قمت بتعريفها داخل قاعدة البيانات مثل أسماء الجداول والأعمدة.

لا يوجد فرق تقني بين كتابة أوامر SQL بأحرف كبيرة أو صغيرة، لكن من الأفضل كتابتها بأحرف كبيرة لتمييزها عن باقي الكلمات داخل الاستعلام

أما الأسماء التي قمنا نحن بتعريفها مثل اسم قاعدة البيانات أو أسماء الجداول، فيجب كتابتها تماماً كما تم تعريفها.
فعلى سبيل المثال، إذا قمت بإنشاء قاعدة بيانات باسم sham فمن الأفضل التعامل معها بنفس الصيغة sham وليس Sham أو SHAM حتى وإن كانت بعض أنظمة إدارة قواعد البيانات لا تفرق بين الحالتين.

Types of SQL Commands (أنواع أوامر SQL)

نوع الأمر	الوصف والأمثلة
DDL Data Definition Language	تستخدم لإنشاء وتعديل هيكل قاعدة البيانات (الجدول). الأوامر: Create, Alter, Drop, Truncate, Rename
DQL Data Query Language	تستخدم لاسترجاع البيانات من الجداول. الأوامر: Select
DML Data Manipulation Language	تستخدم لإدارة البيانات (إضافة، تعديل، حذف) داخل الجداول. الأوامر: Insert, Update, Delete
DCL Data Control Language	تستخدم للتحكم في صلاحيات الوصول والمستخدمين. الأوامر: Grant, Revoke
TCL Transaction Control Language	تستخدم لإدارة العمليات وتأكيد الحفظ أو التراجع. الأوامر: Commit, Rollback, Savepoint

تعليمية SELECT

تستخدم لاسترجاع البيانات من قاعدة البيانات.

الشكل العام لتعليمية SELECT:

```
SELECT column1, column2 FROM table_name  
WHERE condition ORDER BY column;
```

جلب كل الحقول:

```
SELECT * FROM Authors;
```

جلب أعمدة محددة:

```
SELECT au_fname , au_lname FROM Authors;
```

Alias (الأسماء المستعارة):

```
SELECT au_fname AS FirstName,  
       au_lname AS LastName  
From Authors;
```

```
Select au_fname + ' ' + au_lname AS FullName From Authors;
```

```
Select au_Fname + ' ' + au_lname AS FullName,  
       phone AS telephone, City  
From Authors;
```

DISTINCT (القيم الفريدة):

```
SELECT type FROM Titles; SELECT DISTINCT type FROM Titles;
```

ORDER BY

4. الترتيب الافتراضي: تصاعدي

SELECT title, price **FROM** Titles **ORDER BY** price **ASC**;

5. الترتيب التنازلي

SELECT title, price **FROM** Titles **ORDER BY** price **DESC**;

6. ترتيب حسب عامودين

SELECT title, type, price **FROM** Titles **ORDER BY** type **ASC**, price **DESC**;

Just be calm down



WHERE

تستخدم لتصفية البيانات وفق شروط محددة، ويمكن أن نستعمل بداخلها أي عبارة منطقية.

مثال لتصفية الأسعار التي تكون فيها $price > 20$:

SELECT * FROM Titles **WHERE** price > 20;

أمثلة على WHERE

SELECT * FROM Titles **WHERE** price > 20;

SELECT * FROM Publishers **WHERE** state = 'CA';

:WHERE (AND, OR, NOT)

المعامل	الاستخدام
AND	يجب تحقق جميع الشروط المذكورة.
OR	يكفي تحقق واحد على الأقل من الشروط.
NOT	(عكس الشرط) يستبعد السجلات التي تحقق الشرط.
BETWEEN	(من ... إلى) يحدد نطاقاً معيناً للقيم.
IN	للتحقق من وجود القيمة ضمن قائمة محددة.

مثال: لتصفية الأسعار التي تحقق الشرط $price > 10$ AND $price < 30$

SELECT * FROM Titles WHERE price > 10 AND price < 30;

SELECT * FROM Titles WHERE type = 'business' OR type = 'psychology';

SELECT * FROM Titles WHERE NOT type = 'business';

■ :WHERE(BETWEEN, IN)

مثال: لنطاق محدد $price \in [10, 20]$ باستخدام BETWEEN

SELECT * FROM Titles WHERE price BETWEEN 10 AND 20;
SELECT * FROM Titles WHERE type IN ('business', 'psychology');

■ :WHERE (IS NULL)

SELECT * FROM Titles WHERE price IS NULL;
SELECT * FROM Titles WHERE price IS NOT NULL;

■ :WHERE (LIKE)

تستخدم الكلمة المفتاحية LIKE ضمن العبارة الشرطية، كشرط لوجود مثيل. غالباً ما تستخدم هذه الكلمة مع إشارة (%). التي تضاف إلى القيمة التي نبحث عن مثيلاتها، كبديل عن أي رقم من الأرقام أو الأحرف.

Description	Wildcard
A substitute for zero or more characters	%
A substitute for exactly one character	_
Any single character in charlist	[charlist]
Any single character not in charlist	[^charlist]

```
SELECT * FROM Authors WHERE au_Fname Like 'an%';
```

```
SELECT * FROM Authors WHERE au_Fname Like '_n%';
```

```
SELECT * FROM Authors WHERE au_Fname Like '[jab]%';
```

```
SELECT * FROM Authors WHERE au_Fname Like '[^jab]%';
```

أمثلة شاملة:

مثال: لتصفية وترتيب أسعار أكبر من 10 price:

```
SELECT title, price, type FROM Titles WHERE price
> 10 ORDER BY type, price DESC;
```

```
SELECT title AS BookTitle, price AS Price, type AS Category FROM Titles
WHERE price BETWEEN 10 AND 25 ORDER BY Category ASC, Price DESC;
```

ترتيب تنفيذ الاستعلام (Query Execution Order)

عند كتابة استعلام يحتوي على اسم مستعار (Alias) في عبارة SELECT واستخدامه في عبارة where، فإن الاستعلام يعطي نتيجة خاطئة. على سبيل المثال الاستعلام التالي خاطئ:

```
SELECT title AS 'BOOK NAME'
FROM titles
WHERE 'BOOK NAME' LIKE '_[A - C]r%';
```

السبب: لأن عملية WHERE تنفذ قبل SELECT، وبالتالي أثناء تنفيذ WHERE لا يكون الـ Alias قد تم إنشاؤه بعد. الترتيب الفعلي لتنفيذ الاستعلام هو:

7. From
8. WHERE
9. SELECT
10. ORDER BY

لماذا هذا الترتيب؟ لتقليل حجم البيانات المسترجعة بناءً على الجبر العلائقي، وتقليل الاعتمادات بين الحقول.

معاملات المجموعات (SQL Set Operators)

تستخدم لدمج نتائج استعلامين أو أكثر في نتيجة واحدة. وتشارك جميعها في شروط أساسية:

- الاستعلام الأول والثاني يجب أن يرجعا نفس عدد الأعمدة.
- الأعمدة يجب أن تملك نفس نوع البيانات (Data Type) بالترتيب (الأول مع الأول، والثاني مع الثاني... إلخ).
- أعمدة النتيجة النهائية تأخذ أسماءها من الاستعلام الأول.

الوظيفة	التعليمة
دمج نتائج استعلامين مع حذف التكرار تلقائياً.	UNION
دمج نتائج استعلامين مع المحافظة على القيم المكررة.	UNION ALL
إظهار القيم المشتركة فقط بين الاستعلامين (ويحذف التكرار تلقائياً).	INTERSECT
إظهار الصفوف الموجودة في الاستعلام الأول وغير الموجودة في الاستعلام الثاني.	EXCEPT / MINUS

أمثلة وبنية الاستعلامات:

- استخدام UNION لمنع التكرار:

```
SELECT city, country FROM employees
UNION
SELECT city, country FROM customers;
```

- استخدام UNION ALL للمحافظة على التكرار:

```
SELECT city, country FROM employees
UNION ALL
SELECT city, country FROM customers;
```

- استخدام INTERSECT لإيجاد المدن المشتركة:

```
SELECT city, country FROM employees
INTERSECT
SELECT city, country FROM customers;
```

■ استخدام EXCEPT لإيجاد المدن الموجودة لدى الموظفين وليس الزبائن:

```
SELECT city, country FROM employees
EXCEPT
SELECT city, country FROM customers;
```

(الدوال في SQL - SQL Functions)

تستخدم الدوال لإجراء عمليات مختلفة على البيانات لتسهيل معالجتها واستخراج النتائج بشكل مرن واحترافي. تكمن أهميتها في أنها تقلل كمية الكود وتسهل معالجة البيانات. تنقسم الدوال إلى نوعين رئيسيين:

- **الدوال على مستوى الصف الواحد (Single Row Functions):** تعمل على صف واحد في كل مرة، وترجع قيمة واحدة لكل صف بشكل مستقل.
- **الدوال التجميعية (Aggregate Functions / Group Functions):** تعمل على مجموعة من الصفوف معاً لإجراء عمليات حسابية عليها وإرجاع قيمة واحدة تمثل النتيجة النهائية للمجموعة.

■ الدوال على مستوى الصف الواحد (Single-Row Functions):

11. الدوال النصية (Character Functions): تتعامل مع البيانات النصية لمعالجتها وتحويلها.

- LOWER (title): تحويل النص إلى أحرف صغيرة.
- UPPER (title): تحويل النص إلى أحرف كبيرة.
- LEFT (title, 9): استخراج أول 9 أحرف من اليسار.
- RIGHT (title, 9): استخراج آخر 9 أحرف من اليمين.
- SUBSTRING (title, 2, 9): اقتطاع جزء من النص يبدأ من الموقع الثاني وبطول 9 أحرف.
- دوال أخرى: RTRIM, LTRIM, REPLACE, CONCAT, LEN.

12. الدوال الرقمية (Numeric Functions): تتعامل مع القيم الرقمية وترجع نتائج رقمية للحسابات أو التقريب.

- CEILING (price): تقريب الرقم إلى العدد الصحيح الأكبر مباشرة.
- FLOOR (price): تقريب الرقم إلى العدد الصحيح الأصغر مباشرة.
- ROUND (price, 1): تقريب السعر إلى منزلة عشرية واحدة.
- ROUND (price, -1): تقريب السعر إلى أقرب عشرة.
- ABS: القيمة المطلقة.



13. دوال التاريخ (Date Functions): تستخدم لاستخراج وتعديل ومعالجة البيانات الزمنية وتواريخ النظام.

- GETDATE () : عرض التاريخ والوقت الحالي للنظام.
- CAST (GETDATE () AS DATE) : تحويل التاريخ والوقت الحالي لعرض التاريخ فقط دون الوقت.
- YEAR (), MONTH (), DAY () : استخراج السنة، الشهر، أو اليوم بشكل منفصل من التاريخ.
- DATEADD (day, 3, GETDATE ()) : إضافة فترة زمنية للتاريخ (مثلاً إضافة 3 أيام، أو شهر month، أو سنة).

○ DATEDIFF : حساب الفارق بين تاريخين.

○ DATENAME : إعادة اسم الجزء المحدد من التاريخ كنص (مثل اسم الشهر).

14. دوال التحويل (Conversion Functions): تستخدم لتحويل البيانات من نوع إلى آخر (مثل رقم إلى نص أو العكس).

○ CAST (expression AS data_type)

○ CONVERT (data_type, expression, style)

■ الدوال التجميعية (Aggregate Functions):

تتجاهل جميع الدوال التجميعية القيم الفارغة NULL تلقائياً باستثناء دالة COUNT (*).

15. الدالة COUNT () : تستخدم لحساب عدد الصفوف والسجلات.

- COUNT (*) : يعد جميع الصفوف حتى لو احتوت على قيم NULL.
- COUNT (column_name) : يعد فقط الصفوف التي تحتوي على قيم غير فارغة في العمود المحدد.
- COUNT (DISTINCT column_name) : يعد القيم الفريدة فقط بدون تكرار.

16. الدالة SUM () : تستخدم لحساب مجموع القيم الرقمية داخل عمود معين وتعمل فقط مع الحقول الرقمية.

ملاحظة: إذا كانت جميع القيم NULL فإن النتيجة تكون NULL.

17. الدالة AVG () : تستخدم لحساب المتوسط الحسابي للقيم الرقمية وتتجاهل قيم NULL.

18. الدالة MAX ()، الدالة MIN () : تستخدم لإيجاد أكبر أو أصغر قيمة داخل العمود. تعمل مع القيم الرقمية، والتاريخية، والنصية (حسب الترتيب الأبجدي).

العبارات الشرطية والتجميع (GROUP BY & HAVING)

:GROUP BY

يُستخدم الأمر لتجميع الصفوف التي تحتوي على نفس القيمة ضمن عمود معين، وذلك بهدف تطبيق التتابع التجميعية عليها لكل مجموعة بشكل مستقل بدلاً من الجدول بأكمله.

قاعدة هامة: عند استخدام GROUP BY، يجب أن تكون جميع الأعمدة المذكورة في عبارة SELECT إما داخل دالة تجميعية أو مذكورة ضمن عبارة GROUP BY.

:HAVING

تستخدم لتصفية النتائج والشرط بعد تنفيذ العمليات التجميعية المجمعة، حيث لا يمكن استخدام الدوال التجميعية داخل عبارة WHERE.

ملاحظة: محاولة كتابة دالة تجميعية في الـ where مثل:

WHERE SUM (price) > 30

ستؤدي إلى خطأ نظامي، والبديل الصحيح هو استخدامها مع HAVING بعد التجميع.

مقارنة بين WHERE و HAVING:

Having	Where
تستخدم بعد عملية التجميع.	تستخدم قبل عملية التجميع.
تعمل على مستوى المجموعات بالكامل.	تعمل على مستوى الصفوف الفردية.
تستخدم الدوال التجميعية (Aggregate Functions) معها بشكل أساسي لتصفية النتائج.	لا يمكن استخدام الدوال التجميعية (Aggregate Functions) معها.

الصيغة العامة للاستعلام التجميعي المشروط:

*SELECT column_name, aggregate_function(column_name)
FROM table_name*

WHERE condition — — أولاً الصفوف تصفية

GROUP BY column_name — — البيانات تجميع

HAVING condition; — — التجميعية الدالة نتيجة على بناء المجموعات تصفية

أمثلة لتصفية المجموعات والشروط الرياضية:

- شرط عدد الكتب في المجموعة يكون أكبر من 2:
 $COUNT(*) > 2$
- شرط مجموع الأسعار للمجموعة أكبر من 30:
 $SUM(price) > 30$
- شرط متوسط الأسعار أكبر من 15:
 $AVG(price) > 15$
- شرط السعر الأعلى في المجموعة يتجاوز 20:
 $MAX(price) > 20$
- شرط السعر الأدنى في المجموعة يتجاوز 5:
 $MIN(price) > 30$

الاستعلامات الفرعية (Subqueries)

تعدّ الاستعلامات الفرعية (Subqueries) من المفاهيم الأساسية والمتقدمة في لغة SQL، حيث تتيح تنفيذ استعلام داخل استعلام آخر بهدف معالجة البيانات بطريقة أكثر دقة ومرونة. تُستخدم هذه التقنية في عمليات المقارنة، والتصفية، وإجراء الحسابات التجميعية، واستخراج البيانات المرتبطة بنتائج استعلامات أخرى.

مفهوم وبنية ال Subquery

الاستعلام الفرعي هو استعلام SQL يكتب داخل استعلام آخر، حيث تستخدم نتيجة الاستعلام الداخلي من قبل الاستعلام الخارجي لتنفيذ عملية معينة. ويتكون من:

6. استعلام خارجي (Main Query)

7. استعلام داخلي: (Subquery) يتم تنفيذه أولاً، ثم تمرر نتيجته للاستعلام الخارجي.

الشكل العام للاستخدام في عبارة WHERE

SELECT columnA, columnB

FROM Table_Name

WHERE columnB = (Subquery);



الشكل العام للاستخدام في عبارة SELECT

SELECT columnA, (Subquery)

;FROM Table_Name

استخدامات الاستعلامات الفرعية

1. الاستعلامات الفرعية مع IN

تُستخدم لفلتر النتائج بناءً على قائمة قيم مسترجعة من استعلام فرعي.

```
SELECT *
FROM publishers
WHERE pub_id IN (
    SELECT pub_id
    FROM titles
    WHERE type = 'business'
);
```

الاستعلام الداخلي يقوم بجلب أرقام الناشرين pub_id الذين لديهم كتب من نوع business
الاستعلام الخارجي يقوم بجلب جميع بيانات هؤلاء الناشرين من جدول publishers

2. الاستعلامات الفرعية مع EXISTS

يُستخدم التعبير EXISTS للتحقق مما إذا كان الاستعلام الفرعي يعيد أي سجلات. يأخذ التعبير القيمة TRUE في حال أُرجع الاستعلام الفرعي سجلاً أو أكثر، والقيمة FALSE في حال لم يرجع أي سجل. يُفضل استخدام SELECT 1 لأن الكلمة لا تهتم بالقيم المعروضة بل بوجود السجلات.

```
SELECT pub_name
FROM publishers p
WHERE EXISTS (
    SELECT 1
    FROM titles t
    WHERE t.pub_id = p.pub_id
    AND t.type = 'Business'
);
```

الاستعلام الداخلي يتحقق إذا كان للناشر الحالي كتاب واحد على الأقل من نوع business
الاستعلام الخارجي يقوم بعرض أسماء الناشرين pub_name من جدول publishers ولكن فقط للناشرين الذين حققوا شرط الاستعلام الداخلي (لديهم كتاب واحد على الأقل من نوع business)

الشرط t.pub_id = p.pub_id يقوم بربط جدول الكتب مع جدول دور النشر لربط السجلات داخلياً وخارجياً.

3. الاستعلامات الفرعية مع ANY

تعني المقارنة مع أي قيمة ضمن النتائج؛ ويكفي أن يتحقق الشرط مع قيمة واحدة على الأقل من القيم المسترجعة.

```
SELECT title, price
FROM titles
WHERE price > ANY (
    SELECT price
    FROM titles
    WHERE type = 'business'
);
```

الاستعلام الداخلي يقوم ب جلب أسعار جميع الكتب التي نوعها business من جدول titles
الاستعلام الخارجي يقوم بعرض عنوان الكتاب وسعره من جدول titles بشرط أن يكون سعر الكتاب أكبر من أي قيمة واحدة على الأقل من الأسعار التي أعادها الاستعلام الداخلي.

4. الاستعلامات الفرعية مع ALL

تعني أن الشرط يجب أن يتحقق مع جميع القيم المسترجعة بالكامل بدون استثناء.

```
SELECT title, price
FROM titles
WHERE price > ALL (
    SELECT price
    FROM titles
    WHERE type = 'business'
);
```

الاستعلام الداخلي يقوم ب جلب أسعار الكتب التي نوعها business من جدول titles
الاستعلام الخارجي يقوم بعرض عنوان الكتاب وسعره من جدول titles بشرط أن يكون سعر الكتاب أكبر من كل الأسعار التي أعادها الاستعلام الداخلي (أي أكبر من جميع القيم وليس بعضها)

5. الاستعلامات الفرعية مع الدوال التجميعية (Aggregation Functions)

تستخدم لمقارنة الحقول بقيم إحصائية مستخرجة مباشرة مثل المتوسط أو الحد الأعلى.

عرض الكتب التي سعرها أعلى من متوسط الأسعار

```
SELECT title, price
FROM titles
;(WHERE price > (SELECT AVG(price) FROM titles
```

عرض أغلى كتاب في قاعدة البيانات

```
SELECT title, price
FROM titles
;(WHERE price = (SELECT MAX(price) FROM titles
```

ربط الجداول (Join Tables)

تستخدم عملية JOIN لربط البيانات بين جدولين أو أكثر اعتماداً على عمود مشترك بينهما. نظراً لأن قواعد البيانات العلائقية تقسم البيانات إلى جداول مستقلة لتقليل التكرار وتحسين التنظيم، يتم استخدام JOIN عند الحاجة لجمع هذه البيانات في استعلام واحد.

أنواع الربط (Join Types)

1. (الجداء الديكارتي CROSS JOIN _ Cartesian Product)

يقوم بإنشاء جميع التركيبات الممكنة بين صفوف الجدولين، حيث يتم ربط كل صف من الجدول الأول بجميع صفوف الجدول الثاني.

```
SELECT t.title, p.pub_name FROM titles t, publishers p
أو باستخدام الكلمة المفتاحية صراحةً:
SELECT t.title, p.pub_name FROM titles t CROSS JOIN publishers p
```

2. (الربط الداخلي) INNER JOIN

هو النوع الأكثر استخداماً، ويظهر الصفوف التي يوجد بينها تطابق فقط بين الجدولين وفقاً للعمود المشترك.

```
SELECT t.title, p.pub_name
FROM titles t INNER JOIN publishers p
;ON t.pub_id = p.pub_id
```

مثال مشروط: عرض الكتب المنشورة في أميركا وسعرها أكبر من 10

```
SELECT t.title, p.pub_name
FROM titles t
INNER JOIN publishers p ON t.pub_id = p.pub_id
WHERE t.price > 10 AND p.country = 'USA';
```

8. الربط بين أكثر من جدولين (INNER JOIN - Multi Tables):

في هذا المثال يتم عرض كل كتاب مع اسم مؤلفه عبر جدول وسيط (junction table) يسمى titleauthor للاستعلام يقوم أولاً بربط جدول titles مع جدول titleauthor باستخدام العمود المشترك title_id ثم يقوم بربط الناتج مع جدول authors باستخدام العمود المشترك au_id

عرض كل كتاب مع اسم مؤلفه عبر جدول وسيط

```
SELECT t.title, a.au_fname, a.au_lname
FROM titles t
INNER JOIN titleauthor ta ON ta.title_id = t.title_id
INNER JOIN authors a ON a.au_id = ta.au_id;
```

9. (الربط الخارجي) OUTER JOIN

يُستخدم عندما نريد إظهار الصفوف المتطابقة وغير المتطابقة بين الجداول، حيث تظهر البيانات التي لا يوجد لها تطابق على هيئة قيمة NULL.

الوصف والاستعلام	النوع
يعرض كل السجلات من الجدول الأيسر، والسجلات المتطابقة من الأيمن (وتظهر قيم الأيمن NULL إن لم تطابق).	LEFT OUTER JOIN
<pre>SELECT t.title, p.pub_name FROM titles t LEFT OUTER JOIN publishers p ON t.pub_id = p.pub_id;</pre>	
يعرض كل السجلات من الجدول الأيمن، والسجلات المتطابقة من الأيسر.	RIGHT OUTER JOIN

```
SELECT t.title, p.pub_name
FROM titles t RIGHT OUTER
JOIN publishers p ON t.pub_id
= p.pub_id;
```

يعرض كل السجلات من كلا الجدولين سواء كانت متطابقة أو غير متطابقة.

```
SELECT t.title, p.pub_name
FROM titles t FULL OUTER JOIN
publishers p ON t.pub_id =
p.pub_id;
```

FULL OUTER JOIN

لغة تعريف ومعالجة البيانات (DDL & DML)

1. لغة تعريف البيانات (DDL - Data Definition Language)

الأوامر المستخدمة لإنشاء وتعديل وهدم هيكلية الكائنات (مثل الجداول وقواعد البيانات) وتتضمن:

3. لإنشاء كائنات جديدة. CREATE

4. ALTER: لتعديل هيكل الكائن (إضافة/حذف أعمدة أو قيود).

5. DROP: لحذف كائن بالكامل من قاعدة البيانات.

■ سلامة البيانات والقيود (Constraints)

الوظيفة والهدف	القيود
ضمان عدم ترك الحقل فارغاً أثناء إدخال البيانات.	NOT NULL
ضمان عدم تكرار القيم في العمود الواحد.	UNIQUE
المفتاح الأساسي المعرف الفريد للسجل (يجمع تلقائياً بين UNIQUE و NOT NULL).	PRIMARY KEY

التحقق من شرط منطقي معين عند الإدخال مثل: $col_b > 0$	CHECK
تحديد قيمة تلقائية للحقل في حال عدم إدخال قيمة له.	DEFAULT
المفتاح الخارجي لربط جدول بجدول آخر والحفاظ على التكامل المرجعي.	FOREIGN KEY

■ أمثلة وبنية تعليمات DDL

إنشاء وحذف قاعدة بيانات

```
CREATE DATABASE CompanyDB;
```

```
DROP DATABASE CompanyDB;
```

إنشاء وحذف جدول بسيط

```
CREATE TABLE Debt(
    Deptid int NOT NULL,
    Deptname varchar(50) NOT NULL
);
;DROP TABLE Debt
```

■ تعديل الهيكل وإضافة القيود باستخدام ALTER TABLE

إضافة عمود جديد للجدول

```
ALTER TABLE Debt ADD col_b int NOT NULL;
```

تعديل نوع البيانات أو خصائص عمود قائم

```
ALTER TABLE TableName ALTER COLUMN ColumnName NewDataType;
```

حذف عمود من الجدول

```
ALTER TABLE TableName DROP COLUMN ColumnName;
```

(Primary Key) إضافة مفتاح أساسي

ALTER TABLE Debt ADD CONSTRAINT PK_Debt PRIMARY KEY (Deptid);
 إضافة قيد (القيمة الفريدة) (Unique)

ALTER TABLE Debt ADD CONSTRAINT Un_Debt UNIQUE (Deptname);

إضافة قيد التحقق (Check)

ALTER TABLE Debt ADD CONSTRAINT Ch_Debt CHECK (col_b > 0);

إضافة مفتاح خارجي (Foreign Key) (لربط جدول Debt بجدول Emp)

ALTER TABLE Emp ADD CONSTRAINT FK_Emp FOREIGN KEY (Deptid) REFERENCES Debt (Deptid);

حذف قيد معين

ALTER TABLE Debt DROP CONSTRAINT Ch_Deb;

2. لغة معالجة البيانات (DML - Data Manipulation Language)

الأوامر المستخدمة للتعامل مع البيانات الفعلية وإدارتها داخل الجداول بعد إنشائها:

1. INSERT: لإضافة بيانات وسجلات جديدة.

2. UPDATE: لتعديل وتحديث بيانات قائمة بناءً على شرط.

3. DELETE: لإزالة سجلات معينة مع استخدام شرط WHERE لتفادي حذف الجدول كاملاً.

■ أمثلة وبنية تعليمات DML

■ إضافة سجلات جديدة للجداول

INSERT INTO Debt VALUES (1, 'Sales', 1);
 ;(1, 'INSERT INTO Emp VALUES (1, 'Ahmad

تحديث قيمة حقل معين لسجل محدد

UPDATE Emp
 SET Empname = 'samer'
 ;WHERE Empid = 1

حذف سجل محدد من الجدول

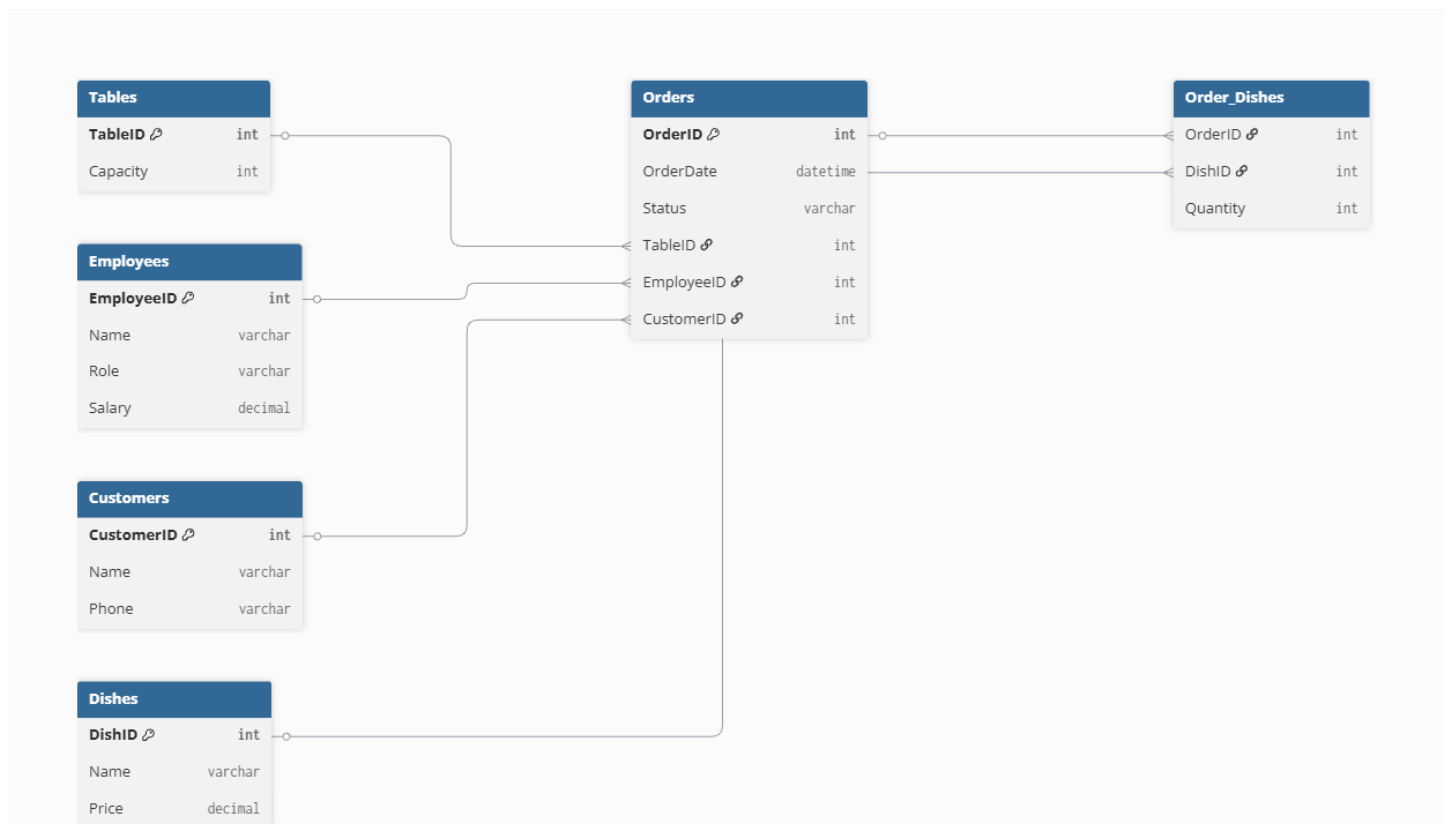
```
DELETE FROM Emp
WHERE Empid = 1;
```

ERD

مسألة:

يرغب مطعم في إنشاء نظام لإدارة الطلبات والعملاء والموظفين والطاولات، يحتوي المطعم على عدد من الطاولات، ولكل طاولة رقم وسعة (عدد الأشخاص) كل طلب طعام في المطعم يكون من طاولة واحدة. يعمل في المطعم عدد من الموظفين، ولكل موظف رقم، واسم، ووظيفة (نادل، طبخ، مدير)، وراتب. يقوم النادل باستقبال الطلبات من العملاء. يقوم العملاء بإنشاء طلبات، حيث يحتوي كل طلب على رقم وتاريخ، وحالة (جديد، قيد التحضرت، مكتمل). يحتوي الطلب على مجموعة من الأطباق، ولكل طبق رقم، واسم، وسعر.

الحل:



الشرح:

يعتمد تصميم قاعدة البيانات لهذا المطعم على تحليل المتطلبات لاستخراج الكيانات (Entities)، والخصائص (Attributes)، والعلاقات (Relationships) بينها وفقاً لمفاهيم النمذجة الكينونية العلائقية (Entity-Relationship Model).

6. الكيانات والخصائص (Entities & Attributes)

تم استنباط الكيانات التالية من وصف المسألة:

- كيان الطاولة (Tables): يمثل الطاولات المتاحة في المطعم. الخصائص: رقم الطاولة (مفتاح أساسي)، والسعة.
- كيان الموظف (Employees): يمثل طاقم العمل. الخصائص: رقم الموظف (مفتاح أساسي)، الاسم، الوظيفة، والراتب.
- كيان العميل (Customers): لم يُذكر صراحة خصائص العميل في المسألة، ولكن بناءً على مبادئ تصميم النظم (لأن العميل "يقوم بإنشاء طلبات")، تم افتراض كيان للعميل يحتوي على: رقم العميل (مفتاح أساسي)، والاسم، ورقم الهاتف.
- كيان الطلب (Orders): يمثل حركات البيع. الخصائص: رقم الطلب (مفتاح أساسي)، التاريخ، والحالة. بالإضافة إلى مفاتيح أجنبية لربط الطلب بالطاولة، والعميل، والنادل.
- كيان الطبق (Dishes): يمثل قائمة الطعام. الخصائص: رقم الطبق (مفتاح أساسي)، الاسم، والسعر.

7. العلاقات (Relationships & Cardinalities)

- علاقة الطاولة بالطلبات (One-to-Many):
الطاولة الواحدة يمكن أن يرتبط بها عدة طلبات (عبر فترات زمنية مختلفة).
الطلب الواحد يصدر من طاولة واحدة فقط (كما نصت المسألة).
- علاقة الموظف (النادل) بالطلبات (One-to-Many):
النادل الواحد يقوم باستقبال عدة طلبات.
الطلب الواحد يتم استقباله وتسجيله بواسطة نادل واحد.

■ علاقة العميل بالطلبات (One-to-Many):
العميل الواحد يمكنه إنشاء عدة طلبات.
الطلب الواحد يعود لعميل واحد.

■ علاقة الطلبات بالأطباق (Many-to-Many):
الطلب الواحد يحتوي على مجموعة من الأطباق.
الطبق الواحد يمكن أن يكون مطلوباً في عدة طلبات مختلفة.

ملاحظة:

قمنا بإنشاء جدول وسيط باسم Order_Dishes لتمثيل علاقة (Many-to-Many). يحتوي هذا الجدول على المفتاح الأجنبي للطلب والمفتاح الأجنبي للطبق، بالإضافة إلى حقل إضافي وهو الكمية (Quantity) ليعبر عن عدد النسخ المطلوبة من نفس الطبق داخل الطلب المعين.

ملاحظة:

بالنسبة للحقول ذات القيم المحدودة مثل Role في جدول Employees و Status في جدول Orders، يمكن تمثيلها بعدة طرق وفقاً لمتطلبات النظام ومستوى التوسع المتوقع:

1. تخزينها مباشرة داخل الجدول باستخدام نوع بيانات VARCHAR.
2. استخدام ENUM عندما تكون القيم معروفة وثابتة، مثل:
Role: Manager, Chef, Waiter
Status: New, Preparing, Completed
3. إنشاء جداول مستقلة مثل Roles و Order_Status وربطها بالجدول الأخرى باستخدام Foreign Keys.

مميزات كل طريقة:

■ تمتاز الطريقة الأولى (VARCHAR) بالبساطة وسهولة التنفيذ، إلا أنها قد تؤدي إلى إدخال قيم غير صحيحة أو غير موحدة (مثل اختلاف التهجئة أو استخدام قيم غير متوقعة)، كما تجعل إدارة القيم وتعديلها أكثر صعوبة مع نمو النظام.

■ وتعد الطريقة الأخيرة أكثر توافقاً مع مبادئ Normalization؛ لأنها تقلل تكرار البيانات، وتحسن سلامة البيانات، وتوفر مرونة أكبر عند إضافة أو تعديل القيم مستقبلاً، خاصة في الأنظمة الكبيرة والقابلة للتوسع.

قسم الاختبارات

1. ما هو النوع الذي تُخزَّن فيه البيانات في قواعد البيانات العلائقية (RDB)؟

ملفات نصية.	A	مصفوفات.	C
جداول (Tables).	B	قواميس.	D

الإجابة: b.

2. ما الاسم الآخر للصفوف (Rows) في قاعدة البيانات العلائقية؟

Fields.	A	Attributes.	B
Tuples.	C	Cells.	D

الإجابة: c.

3. ما الاسم الآخر للأعمدة (Columns) في قاعدة البيانات العلائقية؟

Records.	A	Tuples.	B
Values.	C	Attributes.	D

الإجابة: d.

4. ما تعريف الـ Cell في قاعدة البيانات العلائقية؟

اسم الجدول.	A	تقاطع صف مع عمود.	B
مجموعة من الجداول.	C	نوع البيانات.	D

الإجابة: b.

5. أي من أنواع أوامر SQL يُستخدم لاسترجاع البيانات فقط؟

.DML	B	.DDL	A
.DCL	D	.DQL	C

الإجابة: c.

6. أي أمر SQL يُستخدم لإنشاء جدول جديد؟

.SELECT	B	.INSERT	A
.GRANT	D	.CREATE	C

الإجابة: c.

7. أي مجموعة من الأوامر تنتمي إلى DML؟

.INSERT, UPDATE, DELETE	B	.CREATE, ALTER, DROP	A
.COMMIT, ROLLBACK, SAVEPOINT	D	.SELECT, FROM, WHERE	C

الإجابة: b.

8. ما وظيفة الكلمة المفتاحية DISTINCT في استعلام SELECT؟

إخفاء عمود معين.	B	ترتيب النتائج.	A
تحديد شرط للبحث.	D	إزالة التكرار من النتائج.	C

الإجابة: c.

9. ما الترتيب الافتراضي لـ ORDER BY في SQL؟

تصاعدي (ASC).	B	تنازلي (DESC).	A
عشوائي.	D	أبجدي عكسي.	C

الإجابة: b.

10. أي معامل WHERE يُستخدم للتحقق من نطاق معين من القيم؟

.LIKE	B	.IN	A
.IS NULL	D	.BETWEEN	C

الإجابة: c.

11. ما رمز Wildcard الذي يمثل أي عدد من الأحرف في LIKE؟

%	B	—	A
?	D	*	C

الإجابة: b.

12. ما رمز Wildcard الذي يمثل حرفاً واحداً بالضبط في LIKE؟

%	B	—	A
?	D	*	C

الإجابة: a.

13. ما الترتيب الصحيح لتنفيذ استعلام SELECT في SQL؟

.WHERE → FROM → SELECT → ORDER BY	B	.SELECT → FROM → WHERE → ORDER BY	A
.FROM → SELECT → WHERE → ORDER BY	D	.FROM → WHERE → SELECT → ORDER BY	C

الإجابة: c.

14. لماذا لا يمكن استخدام Alias تم تعريفه في SELECT داخل WHERE؟

لأن WHERE تُنفَّذ قبل SELECT فلا يكون الـ Alias موجوداً بعد.	B	لأن Alias محجوز للجداول فقط.	A
لأن WHERE لا تقبل نصوصاً.	D	لأن SQL لا تدعم Alias في WHERE.	C

الإجابة: b.

15. ما وظيفة معامـل UNION في SQL؟

إيجاد القيم المشتركة فقط.	A	دمج نتائج استعلامين مع إزالة التكرار.	B
دمج نتائج استعلامين مع الإبقاء على التكرار.	C	طرح نتائج استعلام من آخر.	D

الإجابة: b.

16. ما الفرق بين UNION و UNION ALL؟

UNION يدمج جداول مختلفة و UNION ALL يدمج أعمدة.	A	UNION يُزيل التكرار و UNION ALL يُبقي على التكرار.	B
UNION للنصوص و UNION ALL للأرقام.	C	لا فرق بينهما.	D

الإجابة: b.

17. أي معامـل مجموعات يُظهر الصفوف في الاستعلام الأول غير الموجودة في الثاني؟

UNION	A	INTERSECT	B
EXCEPT / MINUS	C	UNION ALL	D

الإجابة: c.

18. أي شرط أساسي يجب توفره عند استخدام معامـلات المجموعات (Set Operators)؟

أن تكون الجداول في نفس قاعدة البيانات.	A	أن يرجع الاستعلامان نفس عدد الأعمدة بنفس أنواع البيانات.	B
أن تكون الجداول خالية من NULL.	C	أن يحتوي الاستعلامان على ORDER BY.	D

الإجابة: b.

19. ما الدالة النصية التي تحوّل النص إلى أحرف صغيرة؟

UPPER()	A	LEFT()	B
LOWER()	C	LEN()	D

الإجابة: c.

20. ما الدالة الرقمية التي تقرب الرقم إلى العدد الصحيح الأكبر مباشرة؟

ROUND()	B	FLOOR()	A
CEILING()	D	ABS()	C

الإجابة: d.

21. ما الدالة التي تعرض التاريخ والوقت الحاليين في SQL Server؟

NOW ().	B	SYSDATE ().	A
CURDATE ().	D	GETDATE ().	C

الإجابة: c.

22. ماذا تعيد دالة COUNT (*) عند وجود صفوف تحتوي على NULL؟

تعيد NULL.	B	تتجاهل الصفوف التي تحتوي على NULL.	A
تعيد خطأ.	D	تعد جميع الصفوف حتى لو احتوت على NULL.	C

الإجابة: c.

23. ما الفرق بين WHERE وHAVING؟

WHERE تُصفي قبل التجميع و HAVING تُصفي بعد التجميع.	B	WHERE للنصوص و HAVING للأرقام.	A
لا فرق بينهما، كلاهما يؤدي نفس الوظيفة.	D	HAVING تُصفي قبل التجميع و WHERE بعده.	C

الإجابة: b.

24. أي مما يلي خطأ في SQL؟

SELECT type FROM Titles WHERE price > 10	B	SELECT type, COUNT (*) FROM Titles GROUP BY type	A
--	---	--	---

SELECT type FROM Titles HAVING COUNT (*) > 2	D	SELECT type FROM Titles WHERE COUNT (*) > 2	C
---	---	--	---

■ الإجابة: c.

25. ما تعريف الاستعلام الفرعي (Subquery) ؟

استعلام SQL يكتب داخل استعلام آخر.	B	استعلام يُنفَّذ بعد الاستعلام الرئيسي.	A
استعلام لا يستخدم WHERE.	D	استعلام مخصص للجداول الكبيرة فقط.	C

■ الإجابة: b.

26. ما الترتيب الصحيح لتنفيذ الاستعلام الفرعي؟

يُنفَّذان في نفس الوقت.	B	يُنفَّذ الاستعلام الخارجي أولاً ثم الداخلي.	A
يعتمد على نوع البيانات.	D	يُنفَّذ الاستعلام الداخلي (Subquery) أولاً ثم الخارجي.	C

■ الإجابة: c.

27. ما الكلمة المفتاحية المستخدمة للتحقق مما إذا كان الاستعلام الفرعي يُعيد أي سجلات؟

ANY.	B	IN.	A
EXISTS.	D	ALL.	C

■ الإجابة: d.

28. ما الفرق بين ANY و ALL في الاستعلامات الفرعية؟

ANY يكفي تحقق الشرط مع قيمة واحدة على الأقل، ALL يجب تحققه مع جميع القيم.	B	ANY للمقارنة مع قيمة واحدة فقط و ALL مع كل القيم.	A
لا فرق بينهما.	D	ALL أسرع ANY في التنفيذ.	C

■ الإجابة: b.

29. أي مما يلي يعرض الكتب التي سعرها أعلى من متوسط أسعار جميع الكتب؟

SELECT title FROM titles WHERE price > (SELECT AVG (price) FROM titles).	B	SELECT title FROM titles WHERE price > MAX (price).	A
SELECT title FROM titles HAVING price > AVG (price).	D	SELECT title FROM titles WHERE price > AVG (price).	C

الإجابة: b.

30. ما وظيفة INNER JOIN في SQL ؟

يُظهر جميع الصفوف من الجدولين بغض النظر عن التطابق.	B	يُظهر جميع الصفوف من الجدول الأيسر فقط.	A
يُظهر الصفوف غير المتطابقة فقط.	D	يُظهر الصفوف المتطابقة فقط بين الجدولين.	C

الإجابة: c.

31. ما نوع الـ JOIN الذي ينتج الجداء الديكارتي (Cartesian Product)؟

LEFT JOIN.	B	INNER JOIN.	A
FULL OUTER JOIN.	D	CROSS JOIN.	C

الإجابة: c.

32. ما وظيفة LEFT OUTER JOIN؟

يعرض السجلات المتطابقة فقط.	B	يعرض كل السجلات من الجدول الأيمن فقط.	A
يعرض جميع السجلات من كلا الجدولين.	D	يعرض كل السجلات من الجدول الأيسر والسجلات المتطابقة من الأيمن.	C

الإجابة: c.

33. في OUTER JOIN، ماذا تكون قيمة الأعمدة غير المتطابقة؟

.NULL	B	صفر (0).	A
سالب واحد (-1).	D	فارغة (").	C

الإجابة: b.

34. ما الكلمة المفتاحية المستخدمة لتحديد شرط الربط في INNER JOIN؟

HAVING	B	WHERE.	A
USING.	D	ON.	C

الإجابة: c.

35. أي أمر DDL يُستخدم لتعديل هيكل جدول موجود؟

MODIFY.	B	UPDATE.	A
CHANGE.	D	ALTER.	C

الإجابة: c.

36. ما القيد (Constraint) الذي يجمع تلقائياً بين NOT NULL و UNIQUE؟

DEFAULT.	B	CHECK.	A
PRIMARY KEY.	D	FOREIGN KEY.	C

الإجابة: d.

37. ما وظيفة القيد FOREIGN KEY؟

ضمان عدم تكرار القيم.	B	منع القيم الفارغة في العمود.	A
تحديد قيمة تلقائية عند عدم الإدخال.	D	ربط جدول بجدول آخر والحفاظ على التكامل المرجعي.	C

الإجابة: c.

38. ما أمر DML المستخدم لإضافة سجلات جديدة إلى جدول؟

APPEND.	B	ADD.	A
CREATE.	D	INSERT INTO.	C

الإجابة: c.

39. أي مما يلي هو الاستخدام الصحيح لـ UPDATE في SQL؟

UPDATE Emp SET Empname = 'samer' WHERE Empid = 1.	B	UPDATE Emp VALUES (1, 'samer').	A
MODIFY Emp SET Empname = 'samer' WHERE Empid = 1.	D	ALTER Emp SET Empname = 'samer'.	C

الإجابة: b.

40. ما الخطر الرئيسي من استخدام DELETE بدون WHERE؟

يُعيد تسمية الجدول.	B	لا يوجد خطر.	A
يوقف قاعدة البيانات.	D	يحذف جميع سجلات الجدول بالكامل.	C

الإجابة: c.

41. ما المشكلة التي تحدث عند تخزين البيانات في أكثر من ملف أو أكثر من برنامج في File System؟

Data Redundancy.	B	Difficult Data Access.	A
Integrity Problems.	D	Security Problems.	C

الإجابة: b.

42. ما هو الفرق الأساسي بين RDBMS و NoSQL؟

NoSQL يستخدم SQL و جداول، بينما RDBMS يستخدم هياكل مرنة بدون جداول صارمة.	B	RDBMS أحدث من NoSQL.	A
RDBMS لا يدعم العلاقات.	D	NoSQL أسرع دائماً من RDBMS.	C

الإجابة: b.

43. ما تعريف نظام إدارة قواعد البيانات (DBMS)؟

برنامج لتخزين الصور فقط.	B	لغة برمجة لإنشاء المواقع.	A
أداة لتصميم واجهات المستخدم.	D	نظام أو برنامج لإنشاء وإدارة والتحكم في قواعد البيانات.	C

الإجابة: c.

44. ما وظيفة مخطط ERD؟

تمثيل وتصميم قواعد البيانات بصرياً قبل التنفيذ.	B	كتابة أوامر SQL تلقائياً.	A
إدارة صلاحيات المستخدمين.	D	تخزين البيانات على الخادم.	C

الإجابة: b.

45. أي نوع علاقة تصف ارتباط طالب بعدة مواد وارتباط كل مادة بعدة طلاب؟

(واحد لمتعدد) 1:M.	B	(واحد لواحد) 1:1.	A
لا يوجد علاقة.	D	(متعدد لمتعدد) M:M.	C

الإجابة: c.

46. ما وظيفة عملية الاختيار (Select - σ) في الجبر العلاقي؟

تغيير اسم الجدول.	B	اختيار أعمدة معينة من الجدول.	A
دمج جدولين ببعضهما.	D	فلتر صفوف الجدول بناءً على شرط منطقي.	C

الإجابة: c.

47. ما وظيفة عملية الإسقاط (Project - Π) في الجبر العلاقي؟

اختيار أعمدة معينة فقط من الجدول وإهمال الباقي.	B	فلتر الصفوف بناءً على شرط.	A
تغيير اسم الجدول أو الأعمدة.	D	دمج جدولين بناءً على حقل مشترك.	C

الإجابة: b.

48. في عملية الاتحاد (U) بالجبر العلاقي، ماذا يحدث عند ظهور سجل مكرر في الجدولين؟

يُحذف بالكامل.	B	يظهر مرتين في النتيجة.	A
يُعطى أولوية للجدول الأول.	D	يظهر مرة واحدة فقط (يُزال التكرار تلقائياً).	C

الإجابة: c.

49. ما الشرط الأساسي لإجراء عمليات المجموعات (Union Compatibility) في الجبر العلاقي؟

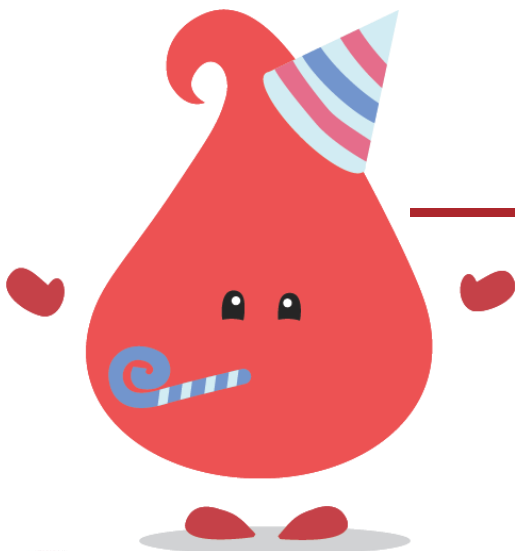
تساوي عدد الأعمدة وتوافق أنواع البيانات في الأعمدة المتقابلة.	B	أن يكون الجدولان في نفس قاعدة البيانات.	A
أن يكون الجدولان من إنشاء نفس المستخدم.	D	أن يكون الجدولان خاليين من القيم NULL.	C

الإجابة: b.

50. ما التعبير الجبري الصحيح لاستخراج أسماء الموظفين في قسم IT الذين يديرون مشروعاً؟

$\Pi_{Name}(\sigma_{dept = 'IT'}(Employee \bowtie EID = Manager Project)).$	B	$\sigma_{dept = 'IT'}(\Pi_{Name}(Employee \bowtie Project)).$	A
$\Pi_{Name}(Employee) \cup \sigma_{dept = 'IT'}(Project).$	D	$\sigma_{Name}(\Pi_{dept = 'IT'}(Employee \bowtie Project)).$	C

الإجابة: b.



مبروك عليك/ي

ربحت/ي ١٠ مليون دولار

51. ما نتيجة تنفيذ هذا الاستعلام؟

SELECT COUNT() FROM Employees WHERE dept = 'IT';*
(الجدول يحتوي 3 موظفين في IT و 5 في HR و 2 في IT بقيمة NULL في salary)

- (a) 5.
- (b) 3.
- (c) 2.
- (d) يُعطي خطأ.

■ الإجابة: b.

52. ما نتيجة هذا الاستعلام إذا كانت قيمة price في أحد السجلات NULL؟

SELECT AVG(price) FROM Titles;

- (a) تُعيد NULL كاملاً.
- (b) تحسب المتوسط، وتتجاهل قيم NULL تلقائياً.
- (c) تُعيد صفر بدل NULL.
- (d) تُعطي خطأ.

■ الإجابة: b.

53. ما الخطأ في هذا الاستعلام؟

SELECT type, COUNT () FROM Titles WHERE COUNT(*) > 2 GROUP BY type;*

- (a) WHERE لا تعمل مع GROUP BY.
- (b) لا يمكن استخدام COUNT (*) داخل WHERE، يجب HAVING.
- (c) COUNT (*) خاطئ، يجب COUNT (type).
- (d) لا يوجد خطأ.

■ الإجابة: b.

ملاحظة:

لأن WHERE تُنفذ قبل GROUP BY وقبل حساب الدوال التجميعية (COUNT, SUM, AVG, ...).
عندما نصل بالتنفيذ إلى WHERE لم يكن قد حسب COUNT(*) أصلاً، لذلك لا يمكن استخدامها هناك.
فنستخدم HAVING.

54. ما نتيجة الاستعلام التالي؟

```
SELECT type, COUNT(*) FROM Titles GROUP BY type HAVING COUNT(*) > 2
```

- (a) يعرض كل الأنواع مع عدد كتبها.
- (b) يعرض الأنواع التي عدد كتبها أكبر من 2 فقط.
- (c) يعرض الكتب التي سعرها أكبر من 2.
- (d) يُعطي خطأ.

■ الإجابة: b.

55. ما نتيجة هذا الاستعلام؟

```
SELECT UPPER('hello') + ' ' + LOWER('WORLD');
```

- (a) .hello world
- (b) .HELLO world
- (c) .hello WORLD
- (d) يُعطي خطأ لأن + لا تعمل مع دوال.

■ الإجابة: b.

56. ما القيمة التي يُعيدها هذا التعبير؟

```
SELECT CEILING(4.1), FLOOR(4.9);
```

- (a) 4 و 5
- (b) 5 و 4
- (c) 5 و 5
- (d) 4 و 4

■ الإجابة: a.

57. في الاستعلام التالي، أي عمود يُعطي خطأ؟

```
SELECT title AS BookName FROM Titles WHERE BookName LIKE 'A%';
```

(a) .title

- (b) BookName في WHERE .
- (c) .LIKE
- (d) لا يوجد خطأ.

■ الإجابة: b.

ترتيب العمليات يكون:

- 1) FROM
- 2) WHERE
- 3) GROUP BY
- 4) HAVING
- 5) SELECT
- 6) ORDER BY

فالخطأ أن الاسم المستعار: (Alias)

title AS BookName

يتم إنشاؤه في مرحلة .SELECT
لكن WHERE تنفذ قبل SELECT

58. ما نتيجة الاستعلام التالي إذا كانت جميع قيم price هي NULL؟

SELECT SUM(price) FROM Titles;

- (a) 0.
- (b) .NULL
- (c) يُعطي خطأ.
- (d) العدد الكلي للمصفوف.

■ الإجابة: b.

59. ما الفرق بين نتيجتي الاستعلامين؟

(1). *SELECT DISTINCT city FROM Customers;*

(2). *SELECT city FROM Customers;*

(a) لا فرق، كلاهما يعطي نفس النتيجة.

(b) يُزيل المدن المكررة، (2) يعرض كل الصفوف.

(c) (2) أسرع دائماً من (1).

(d) (1) يعرض كل الصفوف، (2) يُزيل التكرار.

الإجابة: b.

60. ما ترتيب النتائج في الاستعلام التالي؟

SELECT title, type, price FROM Titles ORDER BY type ASC, price DESC;

(a) ترتيب تصاعدي حسب السعر فقط.

(b) ترتيب تصاعدي حسب النوع، وداخل كل نوع تنازلي حسب السعر.

(c) ترتيب تنازلي حسب النوع والسعر معاً.

(d) ترتيب عشوائي.

الإجابة: b.

61. أيّ استعلام يعرض الكتب التي سعرها بين 10 و20 شاملاً الحدين؟

(a) *WHERE price > 10 AND price < 20*

(b) *WHERE price >= 10 AND price <= 20*

(c) *WHERE price BETWEEN 10 AND 20*

(d) b و c كلاهما صحيح

الإجابة: d.

62. ما نتيجة هذا الاستعلام؟

*SELECT * FROM Authors WHERE au_fname LIKE '[jab]%';*

(a) أسماء تبدأ بـ j أو a أو b

(b) أسماء تحتوي على j أو a أو b في أي مكان

(c) أسماء تنتهي بـ j أو a أو b

(d) أسماء طولها 3 أحرف فقط

الإجابة: a.

63. ما نتيجة هذا الاستعلام؟

```
SELECT pub_name FROM publishers p
WHERE EXISTS (SELECT 1 FROM titles t WHERE t.pub_id =
p.pub_id AND t.type = 'business');
```

(a) جميع دور النشر بغض النظر عن الكتب.

(b) أسماء الناشرين الذين لديهم على الأقل كتاب واحد من نوع business .

(c) كتب من نوع business فقط.

(d) يُعطي خطأ لأن SELECT 1 خاطئة.

الإجابة: b.

64. ما الفرق بين الاستعلامين؟

(1) *WHERE price > ANY (SELECT price FROM T WHERE type = 'biz')*(2) *WHERE price > ALL (SELECT price FROM T WHERE type = 'biz')*

(a) لا فرق بينهما.

(b) (1) سعر أعلى من أي سعر واحد على الأقل، (2) أعلى من جميع الأسعار.

(c) (1) أعلى من جميع الأسعار، (2) أعلى من سعر واحد.

(d) (2) أسرع من (1) دائماً.

الإجابة: b.

65. ما نتيجة هذا الاستعلام إذا كان جدول Titles فارغاً؟

```
SELECT title FROM titles WHERE price > (SELECT AVG (price) FROM titles);
```

(a) يُعطي خطأ.

(b) يُعيد NULL .

(c) لا يُعيد أي صفوف (نتيجة فارغة) .

(d) يُعيد كل الصفوف.

الإجابة: c.

66. لديك جدولان في قاعدة البيانات:
الجدول A يحتوي على 3 صفوف
الجدول B يحتوي على 4 صفوف
عند تنفيذ عملية **CROSS JOIN** بين الجدولين، كم سيكون عدد الصفوف في النتيجة؟

(a) 3.

(b) 4.

(c) 7.

(d) 12.

الإجابة: d.

67. لديك جدولان في قاعدة البيانات:
جدول **Employees** يحتوي على 3 صفوف
جدول **Departments** يحتوي على صفين
عند تنفيذ عملية **INNER JOIN** بين الجدولين باستخدام الشرط dept_id، لا يوجد تطابق لأحد الموظفين كم عدد الصفوف الناتجة من عملية الـ **INNER JOIN**؟

(a) 6.

(b) 5.

(c) 2.

(d) 3.

الإجابة: c.

68. لديك جدولان في قاعدة البيانات
جدول **Titles** يحتوي على 5 كتب
جدول **Publishers** يحتوي على 3 ناشرين
عند تنفيذ عملية **LEFT OUTER JOIN** بين الجدولين، يوجد كتابان لا يملكان ناشرًا مطابقًا. كم عدد الصفوف التي ستعيدها عملية الـ **LEFT OUTER JOIN**؟

(a) 3.

(b) 5.

(c) 7.

(d) 10.

■ الإجابة: b.

69. في استعلام LEFT JOIN، ما قيمة أعمدة الجدول الأيمن للصفوف غير المتطابقة؟

(a) فارغة ("")

(b) 0

(c) NULL.

(d) -1

■ الإجابة: c.

70. أيّ من الاستعلامات التالية يعرض كل كتاب مع اسم مؤلفه عبر جدول وسيط titleauthor؟

(a) FROM titles INNER JOIN authors ON titles.au_id = authors.au_id

(b) FROM titles t INNER JOIN titleauthor ta ON ta.title_id = t.title_id INNER JOIN authors a ON a.au_id = ta.au_id

(c) FROM titles CROSS JOIN authors

(d) FROM titles LEFT JOIN authors ON title = au_fname

■ الإجابة: b.

71. ما الفرق بين FULL OUTER JOIN و INNER JOIN؟

(a) FULL OUTER JOIN أسرع من INNER JOIN.

(b) INNER JOIN يُظهر المتطابقين فقط، FULL OUTER JOIN يُظهر الكل من الجدولين.

(c) FULL OUTER JOIN يُظهر المتطابقين فقط

(d) لا فرق بينهما

■ الإجابة: b.

72. في الاستعلام التالي:

```
FROM titles t INNER JOIN publishers p ON t.pub_id =  
p.pub_id WHERE p.country = 'USA' AND t.price > 10;
```

ما الذي يحدد الربط ويصفي البيانات على التوالي؟

(a) WHERE ثم ON .

(b) ON ثم WHERE .

(c) كلاهما يعملان في نفس الوقت.

(d) ON و WHERE متطابقان في الوظيفة.

■ الإجابة: b.

73. في مخطط ERD ، بأي شكل هندسي تُرسم الكيانات (Entities)؟

(a) دائرة.

(b) معين (رومبوس).

(c) مستطيل.

(d) خط مباشر.

■ الإجابة: c.

74. في مخطط ERD ، بأي شكل هندسي تُرسم الخصائص (Attributes)؟

(a) مستطيل.

(b) دائرة بيضاوية.

(c) معين.

(d) مثلث.

■ الإجابة: b.

75. في مخطط ERD ، بأي شكل هندسي تُرسم العلاقات (Relationships)؟

(a) مستطيل.

(b) دائرة.

(c) معين (رومبوس).

(d) خط منقط.



■ الإجابة: c.

76. كيف يُمَيِّز مفتاح الكيان (Primary Key Attribute) في ERD؟

- (a) بلون أصفر.
- (b) بشكل مستطيل بدلاً من بيضاوي.
- (c) بخط تحته (Underline) داخل الشكل البيضاوي.
- (d) بوضع النجمة (*) قبل اسمه.

■ الإجابة: c.

77. ما الكيان الضعيف (Weak Entity) في ERD؟

- (a) كيان يحتوي على بيانات قليلة.
- (b) كيان لا يمكن تعريفه بشكل فريد بدون الاعتماد على كيان آخر.
- (c) كيان ليس له علاقات مع كيانات أخرى.
- (d) كيان يحتوي على مفتاح خارجي فقط.

■ الإجابة: b.

78. كيف يُرسم الكيان الضعيف (Weak Entity) في ERD؟

- (a) مستطيل عادي.
- (b) دائرة مزدوجة.
- (c) مستطيل مزدوج (double rectangle).
- (d) معين منقط.

■ الإجابة: c.

79. في نظام مستشفى:

- طبيب واحد يمكن أن يعالج عدة مرضى.
- كل مريض يتم علاجه بواسطة طبيب واحد فقط.
- ما نوع العلاقة بين جدول الأطباء (Doctors) وجدول المرضى (Patients) ؟

- (a) متعدد لمتعدد
- (b) واحد لواحد.
- (c) واحد لمتعدد.
- (d) لا توجد علاقة.

■ الإجابة: c.

80. في نظام جامعة:

الطالب الواحد يمكنه التسجيل في عدة مواد.
المادة الواحدة يمكن أن يسجل فيها عدة طلاب.
ما نوع العلاقة بين الطلاب (Students) و المواد (Courses)؟

- (a) 1:1.
- (b) 1:M.
- (c) M:N.
- (d) لا توجد علاقة.

■ الإجابة: c.

81. أي مما يلي مثال على علاقة 1:1 في ERD؟

- (a) قسم ↔ موظفين.
- (b) طالب ↔ مواد.
- (c) شخص ↔ رقم هوية وطنية.
- (d) منتج ↔ طلبات.

■ الإجابة: c.

82. لماذا تُحوّل علاقة M:N إلى جداول إضافية عند التنفيذ الفعلي؟

- (a) SQL لا يمكنها تمثيل علاقة M:N نهائياً بأي شكل.
- (b) لأنه لا يمكن تمثيل M:N مباشرة في جداول علائقية بدون جدول وسيط.
- (c) لأن M:N تستهلك ذاكرة كبيرة.
- (d) لأن DBMS لا يفهم M:N.

■ الإجابة: b.

83. في ERD ، ما الخاصية المتعددة القيم (Multi-valued Attribute)؟

- (a) خاصية لها قيمة واحدة فقط.
- (b) خاصية يمكن أن تحمل أكثر من قيمة لنفس الكيان (مثل: أرقام هاتف متعددة).
- (c) خاصية مشتقة من خصائص أخرى.
- (d) خاصية تُستخدم كمفتاح أساسي.

■ الإجابة: b.

84. كيف ترسم الخاصية المتعددة القيم (Multi-valued Attribute) في ERD؟

- (a) بيضاوية عادية.
- (b) بيضاوية بخط تحتها.
- (c) بيضاوية مزدوجة (double ellipse).
- (d) مستطيل بدلاً من بيضاوية.

■ الإجابة: c.

85. ما الخاصية المشتقة (Derived Attribute) في ERD؟

- (a) خاصية لا يمكن حسابها.
- (b) خاصية قيمتها تحسب من خصائص أخرى (مثل: العمر يُحسب من تاريخ الميلاد).
- (c) خاصية تُستخدم كمفتاح خارجي.
- (d) خاصية تحتوي على قيم متعددة.

■ الإجابة: b.

86. كيف ترسم الخاصية المشتقة (Derived Attribute) في ERD؟

- (a) بيضاوية عادية مملوءة.
- (b) بيضاوية منقطة الحدود (dashed ellipse).
- (c) بيضاوية مزدوجة.
- (d) مستطيل صغير.

الإجابة: b.

87. ما المصطلح المستخدم لوصف أسلوب رسم مخططات ERD باستخدام رموز مثل الخطوط العمودية وأصابع الغراب؟

- .UML Notation (a)
- .UML Notation (b)
- .Crow's Foot Notation (c)
- .Bachman Notation (d)

الإجابة: c.

88. في مخططات العلاقات وقواعد البيانات (ERD) باستخدام أسلوب (Crow's Foot - رجل الغراب)، ما الرمز الذي يُعبّر عن القيد (واحد وواحد فقط - One and only one)؟

- (a) خط عمودي واحد وخط عمودي آخر |— .
- (b) خطان عموديان || .
- (c) ثلاثة خطوط (أصابع الغراب) مع دائرة.
- (d) أصابع الغراب وحدها.

الإجابة: b.

89. في مخططات العلاقات وقواعد البيانات (ERD) باستخدام أسلوب (Crow's Foot - رجل الغراب)، ماذا تعني أصابع الغراب الثلاثة (الشوكة/التفرع - Three lines/Fork) بجانب الكيان؟

- (a) واحد فقط.
- (b) صفر أو واحد.
- (c) كثير (Many).
- (d) صفر أو كثير.

الإجابة: c.

90. في Crow's Foot، ما رمز العلاقة الإلزامية الواحدة (One - mandatory)؟

- (a) دائرة مع خط عمودي.

- (b) خطان عموديان متوازيان || .
- (c) أصابع الغراب مع دائرة.
- (d) خط منقط.

■ الإجابة: b.

91. في Crow's Foot ، ما رمز العلاقة (Zero or One اختياري واحد)؟

- (a) || خطان عموديان.
- (b) O دائرة وخط عمودي.
- (c) أصابع الغراب مع خط عمودي.
- (d) خط عمودي وحيد.

■ الإجابة: b.

92. في Crow's Foot ، ما رمز العلاقة (Zero or Many اختياري كثيرة)؟

- (a) أصابع الغراب مع خط عمودي.
- (b) أصابع الغراب مع دائرة <O .
- (c) خطان عموديان || .
- (d) خط منقط مع دائرة.

■ الإجابة: b.

93. في Crow's Foot ، ما رمز العلاقة (One or Many إلزامية كثيرة)؟

- (a) أصابع الغراب وحدها.
- (b) دائرة مع أصابع الغراب
- (c) أصابع الغراب مع خط عمودي <| .
- (d) خطان عموديان || .

■ الإجابة: c.

94. في مخطط Crow's Foot بين Employee و Department:

طرف Employee: أصابع الغراب مع خط عمودي.

طرف Department: خطان عموديان.

ما نوع العلاقة؟

(a) M:N

(b) 1:1

(c) 1:M قسم واحد يحتوي على موظفين كثيرين، كل موظف في قسم واحد.

(d) M:1

الإجابة: c.

95. في مخطط Crow's Foot بين Student و ID_Card:

كلا الطرفين: خطان عموديان ||

ما نوع العلاقة؟

(a) (واحد لواحد) 1:1

(b) (واحد لمتعدد) M:1

(c) (متعدد لمتعدد) M:M

(d) Zero or Many

الإجابة: a.

96. في Crow's Foot ، ما الفرق بين الدائرة (O) والخط العمودي (|) بجانب الرمز الرئيسي؟

(a) الدائرة تعني إلزامي، الخط يعني اختياري.

(b) الدائرة تعني اختياري (يمكن صفراً)، الخط يعني إلزامي على الأقل واحد.

(c) كلاهما يعطيان نفس المعنى.

(d) الدائرة للنصوص والخط للأرقام.

الإجابة: b.

97. في نظام طلبات: طلب واحد يحتوي على عدة منتجات، ومنتج واحد يمكن أن يكون في عدة طلبات.

كيف ترسم هذه العلاقة في Crow's Foot ؟

(a) || على كلا الطرفين.

- (b) أصابع الغراب مع خط عمودي على كلا الطرفين (M:N) .
- (c) أصابع الغراب على طرف المنتج فقط.
- (d) خط مستقيم بدون أي رموز.

■ الإجابة: b.

98. قاعدة بيانات مكتبة: كاتب واحد يمكنه كتابة صفر أو أكثر من الكتب، وكل كتاب له كاتب واحد إلزامياً. ما الرموز الصحيحة في Crow's Foot ؟

- (a) طرف الكاتب: || ، طرف الكتاب.
- (b) طرف الكتاب (zero or many) $O <$ ، طرف الكاتب (one and only one) || .
- (c) طرف الكتاب (one or many) $| <$ ، طرف الكاتب (zero or one) $O |$.
- (d) طرف الكتاب $O |$ ، طرف الكاتب $O <$.

■ الإجابة: b.

99. في Crow's Foot ، ما أهمية تمييز العلاقة بين 'Zero or Many' و 'One or Many' ؟

- (a) لا فرق في الممارسة العملية.
- (b) يُحدد إن كان وجود السجل اختيارياً أم إلزامياً، ويؤثر على القيود في قاعدة البيانات.
- (c) يُستخدم فقط للتوثيق الجمالي.
- (d) يُحدد نوع البيانات في العمود.

■ الإجابة: b.

100. مخطط Crow's Foot لنظام جامعة:

بين Course و Student : أصابع الغراب مع دائرة على كلا الطرفين

ما نوع العلاقة وماذا تعني الدائرة؟

- (a) 1:1 والدائرة تعني إلزامي
- (b) M:N واختيارية على كلا الطرفين (Zero or Many) طالب قد لا يكون مسجلاً وكورس قد يكون فارغاً
- (c) 1:M والدائرة تعني الطرف الأيسر فقط.
- (d) M:N ولكن الدائرة لا معنى لها في Crow's Foot

■ الإجابة: b.